Kernel Hacking 3 – kbuild e moduli

- Questa è anche l’ultima presentazione per il modulo di kernel hacking dell’insegnamento di Sistemi Operativi della laurea triennale



Preambolo su ambiente di sviluppo

- Prima di iniziare con gli argomenti di questa presentazione, alcuni suggerimenti per aumentare la produttività e stabilità del vostro ambiente di sviluppo

Utilizzo terminale locale all’host

- L’interfaccia principale utilizzata nel kernel hacking è una shell in un terminale
  - E’ molto più pratico utilizzare un terminale locale all’host, che un terminale in esecuzione nell’interfaccia grafica del guest
  - Perché si riesce a rimanere nello stesso ambiente
    - Si utilizza un terminale familiare, con dimensioni svincolate dalla dimensione della finestra dell’emulatore
    - Si ha il copia e incolla
    - Non si deve aspettare l’avvio dell’interfaccia grafica del guest
    - …

Shell remota 1/2

- Per utilizzare un terminale locale, avete due opzioni
  - Utilizzare un emulatore senza interfaccia grafica, ed eseguirlo in un terminale locale
  - Collegarsi al guest tramite ssh, da un terminale locale
- Se si utilizza la seconda opzione
  - E’ necessario che l’emulatore di macchina virtuale abbia una rete interna abilitata (comportamento ormai predefinito)
  - Che nel SO guest sia installato il pacchetto openssh-server

Shell remota 2/2

- Poi dovete scoprire l'indirizzo di rete del guest, all'interno della rete emulata nell'host. Di norma basta un `ifconfig`, oppure un `ip addr show`, da dentro il guest
- Infine non vi resta che utilizzare tale indirizzo col comando ssh
  - Ad esempio, se l’indirizzo fosse 10.0.2.15, dall'host eseguite: `ssh nome_utente@10.0.2.15`

ssh-copy-id

- Ogni volta che vi collegate in ssh dovete reinserire la password
- Potete automatizzare questo passaggio con il comando ssh-copy-id
  - Esempio
    ssh-copy-id nome_utente@10.0.2.15
- Una volta eseguito `ssh-copy-id` non sarà più necessario inserire la password
- Se `ssh-copy-id` non funziona perché non avete una chiave nell'host, allora generatela con `ssh-keygen`

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Build del kernel: kbuild

- Come vengono eseguiti:
  - Creazione dell’immagine del kernel
  - Creazione dei moduli
  - Creazione dell’initramfs
  - Installazione del tutto

    ?
- Attraverso il processo di build del kernel: *kbuild*

Meccanismo di kbuild: make

- Realizzato attraverso il comando make
  `man make`
- Le operazioni eseguite dal comando make sono stabilite attraverso una combinazione di file
  - Un file di configurazione del kernel
  - Un albero di makefile
  - Un albero di file di configurazione

Meccanismo di kbuild: .config

- File di configurazione del kernel
- .config, costituito da una sequenza di righe, in cui ogni riga contiene
  - il nome di una cosiddetta opzione di configurazione, che non è altro che una parola
  - seguita dal simbolo di assegnamento
  - seguito dal valore assegnato alla opzione
- Il .config deve essere presente nell'albero in cui si effettua il build del kernel (ossia in cui vengono generati tutti i file durante il kbuild)
  - Di default, l'albero di build coincide con l'albero dei sorgenti
- Apriamo il .config del kernel che abbiamo compilato, solo per guardare la forma
- Approfondiamo fra un attimo le opzioni

Meccanismo di kbuild: i Makefile

- I Makefile sono organizzati gerarchicamente
- Un top Makefile, presente nella radice dell’albero dei sorgenti
- Un Makefile per ogni sottodirectory dell’albero dei sorgenti. Quasi in ogni caso:
  - La sottodirectory contiene file sorgenti dai quali si possono generare uno o più moduli
  - Il makefile è un kbuild Makefile per tali moduli, ossia contiene le istruzioni (nel linguaggio Makefile) per generare tali moduli
- Apriamo anche il top Makefile ed un qualsiasi kbuild Makefile, solo per farci un’idea superficiale

Meccanismo di kbuild: i Kconfig

- Un albero di file di configurazione che definiscono l’insieme (database) di opzioni di configurazione
- Il Kconfig nella radice dell’albero dei sorgenti si limita a includere tutti Kconfig contenuti in ciascuna delle subdirectory
- Apriamolo per farci un’idea
- I Kconfig contenuti nelle subdirectory sono quelli che effettivamente contengono le definizioni delle opzioni

Cominciamo dall’usare il kbuild

- Per utilizzare il sistema di kbuild, il comando make va sempre eseguito dalla radice dell’albero dei sorgenti
- Partiamo con un
  `make help`
- Tra i target interessanti troviamo quelli di manipolazione del file .config
- Uno dei più usati è `menuconfig`, perché funziona anche tramite *ssh*
- Eseguiamo
  `make menuconfig`
  e facciamoci un giro per farci un’idea concreta

Flusso di lavoro generale

- Se si parte da .config ancora non esistente
  1) Creazione di un file .config
     - Eventuale modifica, utilizzando uno dei target di config del Makefile
  2) Build dei moduli dinamici e dell'immagine del kernel
  3) Installazione dei moduli dinamici
  4) Installazione dell'immagine
  5) Riavvio

- Nel caso di modifica del .config o di modifica dell'immagine del kernel, si ripetono i passi da 2 a 5

- Altrimenti, nel caso di sola modifica di qualche modulo dinamico
  1) Rebuild dei soli moduli dinamici modificati
  2) Rimozione dal kernel dei moduli modificati (lo vedremo)
  3) Ricaricamento dei moduli modificati (lo vedremo)

Domanda

1)Se si modifica il sorgente di qualche parte del core o di qualche modulo built-in del kernel, e poi si ricompila e si reinstalla il kernel, cosa accade alla immagine del kernel presente nella directory `/boot`?

2)Se si modifica il sorgente di qualche modulo dinamico, e poi si ricompila il modulo e si esegue il target `modules_install`, cosa accade al file del modulo nella cartella `/lib/modules/<percorso_rimanente_per_arrivare_al_modulo>`?

Risposta

1)L’immagine in `/boot` viene sovrascritta con la nuova immagine prodotta nel build
2)Il file viene sovrascritto col nuovo `.ko` prodotto nella ricompilazione del modulo

Domanda

- L’immagine del kernel è l’unico file che viene creato/sovrascritto dal target `install` nella directory `/boot`?

Risposta

- No, come spiegato in dettaglio nella prossima slide

File creati in `/boot`

- Il target `make install` crea/sovrascrive quattro file
  - Immagine del kernel: `vmlinux-<versione>` o `vmlinuz-<versione>`
    - Copiato dall'albero di build
      - A breve cerchiamo da dove, esattamente
  - Initramfs: `initrd-<versione>`
    - Generato, ad esempio col comando `mkinitramfs`
  - Config del kernel: `config-<versione>`
    - Copiato dall'albero di build, il file sorgente è il .config
  - Tabella dei simboli: `System.map`
    - Non approfondiamo
- Dopo aver creato/sovrascritto questi file, il make install aggiorna tipicamente il bootloader (eseguendo `update-grub` nel caso di `grub`)

Ricerca immagine compilata

- Trovate l’immagine del kernel che è stata generata nel proprio albero di build e che è stata poi installata nel sistema

Soluzione

- Un possibile metodo è il seguente
  - Cercare tutti i file che hanno *image* nel nome
    `find . -iname "*image*"`
  - Tra tali file, su ciascuno di quelli che vi sembra un candidato a contenere un'immagine del kernel, usare il comando `file` per confermare che si tratta quantomeno di un file binario eseguibile
  - Per ciascuno dei candidati filtrati nel passaggio precedente, usare il comando `diff` per scoprire qual è il candidato giusto
    `diff <immagine-candidata> /boot/vmlinuz-<versione>`
- Nel mio build si tratta di
  `arch/arm64/boot/Image`

Esercizio

- Come controllare che l’immagine del kernel in esecuzione corrisponda all’ultima immagine compilata?

Soluzione

- Con `uname -a` ottenete data e orario di ultima compilazione del kernel
- Confrontate tale orario con l'orario di ultima modifica dell'immagine nel build e/o in `/boot`

Domanda

- Vedremo a beve i dettagli di come si carica e come si rimuove un modulo
- Ma approfondiamo prima un aspetto concettuale importante
- Se un modulo è correntemente caricato nel kernel, allora, quando lo si ricompila e reinstalla, cambia anche la versione del modulo correntemente caricata nel kernel?

Risposta

- No
- Come si è visto, il caricamento di un modulo consiste nel caricamento della sua immagine nello spazio di indirizzamento del kernel, e nella rilocazione dei simboli
- Dovete distinguere tra
  - Il file immagine del modulo, che risiede nel filesystem
  - L'immagine del modulo caricata in memoria principale
- Ecco perché, se si vuole avere anche nel kernel running la versione aggiornata del modulo, bisogna caricarlo di nuovo (dopo averlo prima rimosso, nel caso in cui fosse correntemente già caricato)

Pausa su kbuild

- Sospendiamo qui la sequenza di nozioni sul kbuild, per darvi la possibilità di fare il primo incontro con la creazione di un modulo
  - In particolare un modulo esterno
- Poi torneremo sul kbuild e lo completeremo, prima di passare alla creazione del nostro primo modulo interno

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Descrizione moduli ed operazioni

- Riprendiamo e completiamo la descrizione dei moduli, studiando: https://medium.com/@adilrk/kernel-modules-86f07c93362a
  - Esclusa la sezione *Device Files*
  - Proviamo il comando `lsmod`
  - Capiamo solamente cosa fanno gli altri comandi
    - Avremo modo di provarli a brevissimo

Modulo esterno

- Definiamo come esterno un modulo compilato al di fuori del kernel build
- Anche detto *out-of-tree*
- Tipicamente il progetto di un modulo esterno risiede in una directory esterna all'albero dei sorgenti del kernel
- Dato il kernel per cui si crea un modulo esterno, l'albero dei sorgenti di tale kernel può non essere presente affatto nel sistema in cui si crea un modulo esterno
  - Sono necessari solo gli *header file* di tale kernel
  - Vedremo dove trovarli, più molto altro, con il prossimo esercizio

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Coincidenza file sorgente

- Come già detto, per la compilazione di un modulo esterno bisogna disporre dei sorgenti del kernel per cui si vuole compilare il modulo
- Per il nostro esempio, creeremo il modulo per il nostro kernel custom, assumendo che sia proprio quello in esecuzione
- Affinché tale modulo venga correttamente compilato, i sorgenti devono corrispondere esattamente a quelli che sono stati utilizzati quando abbiamo creato l'immagine del nostro kernel custom presente in `/boot`
- Quasi sicuramente noi potremmo avere perso questa corrispondenza, perché abbiamo creato un'altra immagine per il nostro hack `Hello world`
- Il rischio che corriamo è di avere in esecuzione una versione del kernel custom che non corrisponde esattamente ai file sorgente
- Perché magari è il kernel senza la stampa `Hello world`

Reset dell’albero dei sorgenti

- Un metodo definitivo per evitare il problema è eseguire il seguente comando da dentro l’albero dei sorgenti linux
  `git reset --hard`
  - Questo comando ripristina lo stato di tutti i file a come era prima di qualsiasi nostro cambiamento
- A questo punto ricompiliamo il kernel e reinstalliamo il kernel e i moduli
- Riavviamo ed assicuriamo che il kernel running sia esattamente l’ultimo compilato
  - Per esempio controlliamo la data di compilazione del kernel running, con `uname -a`

Partiamo!

- Riproviamo a seguire lo stesso schema che abbiamo seguito con la prima compilazione del kernel
  - Black box!
- Nelle seguenti slide vi do istruzioni per creare il modulo e per caricarlo
- Non vi preoccupate minimamente se giustamente alcuni dettagli non vi saranno chiari
- Poi vi analizzeremo ogni file ed ogni dettaglio

Sviluppo di un modulo esterno

- Seguite il tutorial
  https://tldp.org/LDP/lkmpg/2.6/html/x121.html
  fino alla sezione 2.5, per creare il modulo ed il Makefile
  - Inoltre, studiate bene anche tutte le informazioni fornite nel testo di ogni sezione, ed eseguite l'esercizio suggerito in fondo alla sezione 2.2
- Anche questo tutorial è un esempio di documentazione obsoleta, nonostante i moduli nel kernel siano un argomento ormai consolidato
  - Lo utilizziamo lo stesso, per darci una prima occasione concreta per sviluppare le nostre capacità di utilizzare anche documentazione imperfetta
  - Per aiutarvi in questa prima esperienza con documentazione imperfetta, vi segnalo in anticipo tutti gli elementi obsoleti di questo tutorial, nelle prossime slide

`init_module()` deprecata

- In fase di compilazione potreste ottenere un errore di *function name deprecated*, per la funzione `init_module()`
  - E' dovuto all'effettivo utilizzo di una modalità ormai deprecata per definire la funzione di inizializzazione di un modulo
  - Per fortuna, nel tutorial stesso viene fornita la versione corretta, nella sezione 2.3
    - Nel tutorial, questa versione corretta viene presentata come una alternativa 'moderna'
      - E' invece l'unica alternativa al momento corretta

Errori di simboli non definiti

- Se la compilazione da errori di simboli non definiti (tipo quello della `printk`), può essere un problema di configurazione del kernel
- Provate a rigenerare il config, ad esempio, con un `defconfig`. Poi ricompilate il kernel

Ultimi dettagli obsoleti

- La mancanza della macro `MODULE_LICENSE()` può far fallire direttamente la compilazione. La `MODULE_DESCRIPTION()` mancante invece dovrebbe generare una *warning*. Trovate dettagli nella sezione 2.5, dove però è affermato erroneamente che la mancanza di tali macro può generare solo *warning* all'atto dell'inserimento del modulo
- Sezione 2.2
  - In alcune distro, può essere necessario essere root per eseguire `modinfo`. Altrimenti `modinfo` potrebbe persino non essere trovato dalla shell, perché l'eseguibile `modinfo` è dentro `/sbin`, e quest'ultimo percorso potrebbe non essere presente nella variabile PATH
- Sezione 2.4
  - Diversamente da quanto sembra indicare il testo nella sezione, l'unica differenza tra `hello-3.c` ed `hello-2.c` è l'aggiunta di `__initdata`

Inizio analisi

- Una volta che il modulo si compila e si carica correttamente, iniziamo con la nostra analisi
- Analizziamo innanzitutto la versione più completa del sorgente del modulo
  - Per chi non è in aula, studiate ed assicuratevi di aver capito tutto ciò che è spiegato nel testo del tutorial
- Passiamo quindi al secondo ed ultimo file: il `Makefile`
  - Per poter effettuare questa analisi, dobbiamo prima completare le nostre conoscenze sul sistema di kbuild

Completamento conoscenze kbuild

- Possiamo ora approfondire e completare le nostre conoscenze
- https://docs.kernel.org/kbuild/index.html
  Diamo solo uno sguardo solo per farci un’idea
- https://docs.kernel.org/kbuild/kconfig-language.html
  Studiamo fino a *“Menu attributes”* inclusa. Diamo uno sguardo al resto.
- https://docs.kernel.org/kbuild/kbuild.html
  Diamo solo uno sguardo
- https://docs.kernel.org/kbuild/makefiles.html
  Studiamo fino alla sezione *"Library file goals - lib-y"* esclusa, più la sola sezione *“Descending down in directories”*

Eccezione del top Makefile

- Rispetto a quanto riportato in *“Descending down in directories”*, il top Makefile fa eccezione, perché non contiene alcuna regola esplicita per la ricorsione nelle subdirectory
- Include invece altri Makefile, che a loro volta cercano, e fanno eseguire ricorsivamente, i Makefile contenuti nelle varie subdirectory
- Non approfondiamo ulteriormente questo aspetto

Analisi Makefile modulo esterno 1/2

- `obj-m += hello_module.o`
  Aggiunge l'obiettivo `hello_module.o` alla stringa nella variabile `obj-m`.
  Concettualmente: dice al sistema di kbuild di aggiungere `hello_module.ko` alla sequenza di moduli dinamici da compilare
- Continua ...

Analisi Makefile modulo esterno 2/2

- `make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules`
  - Invoca ricorsivamente il comando `make`, ma con un diverso Makefile. In particolare, prima di avviare `make`, si sposta nella directory indicata dalla opzione `-C`, quindi in tale directory cerca il nuovo Makefile da eseguire. Lì si trova proprio il Makefile per la versione del kernel indicata da `uname -r`, nonché tutti gli header file di tale kernel.
  - Per l'esecuzione ricorsiva di tale Makefile, assegna `$(PWD)` alla variabile `M`, il che fa limitare il build ai soli moduli presenti nella directory `$(PWD)`. Come target, esegue il target `modules`, per generare appunto i moduli (stesso risultato se non si aggiungeva `modules` affatto)
- `make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean`
  Tutto come il precedente, ma esegue il target `clean`

Controllo caricamento modulo

- Come controllate se un modulo è stato caricato?
  - Oltre al metodo suggerito nel tutorial

Risposta

- Eseguendo `lsmod`
- O anche controllando i messaggi con `dmesg`, nel caso in cui il modulo scriva qualcosa sul kernel log al caricamento

Esercizio

- Modificate il messaggio di saluto del modulo, e collaudate la nuova versione
- Che operazioni bisogna effettuare per avere la nuova versione del modulo caricata?

Modifica modulo

- Dopo aver modificato il sorgente, bisogna
  - Ricompilare
  - Eseguire `rmmod` se il modulo era già caricato
  - Eseguire `insmod` per caricare la nuova versione
- Quindi NON basta ricompilare il modulo

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Riprendiamo il kbuild

- Diamo uno sguardo di nuovo ad un esempio di Makefile e Kconfig, per rinfrescarci le idee
- Poi vediamo alcune modalità di invocazione del comando make, per restringere l’insieme dei file da compilare

Esempio

- Per esempio, guardiamo tali file in `drivers/soc/microchip`
- Troviamo poi
  - L'istruzione, nel Makefile e nel Kconfig in drivers/soc, che fa avvenire la ricorsione
  - La stessa cosa nel Makefile e nel Kconfig in drivers
    - Sia questo che l'altro Makefile sono configurati per entrare sempre nella ricorsione, attraverso `obj-y`
  - Poi nel top Kconfig
  - Non c'è nulla invece nel top Makefile

Ricompilazione dei soli moduli

- Nel caso in cui si sia certi che, dall'ultimo kbuild, si è modificato solo il codice sorgente di qualche modulo, ed in un'unica subdirectory, allora si può ricompilare in modo ancora più veloce, grazie alla seguente funzionalità
- Per ricompilare solo i moduli eventualmente presenti in una subdirectory:
  `make M=<percorso-subdirectory>`
- Quindi il flusso di lavoro abbreviato, per il caso speciale di solo moduli modificati e solo in una subdir, diviene:

Flusso di lavoro con `M=`

1)Rebuild del modulo e dei moduli dinamici presenti nella subdirectory interessata:
`make M=<percorso-subdirectory>`

2)Installazione dei moduli dinamici

3)Rimozione dal kernel dei moduli modificati (lo vedremo)

4)Ricaricamento dei moduli modificati (lo vedremo)

Limitazioni dell’opzione `M=`

- Assume che la configurazione del kernel non sia stata modificata
- Se si è modificata la configurazione, la correttezza del build non è garantita
- Nel caso in cui si sia modificata anche la configurazione, oppure nel caso in cui ci siano anche moduli built-in da compilare in una subdirectory, si può utilizzare la seguente alternativa

Compilazione di una sola subdirectory

`make <subdirectory in cui ci sono i file di cui volete effettuare la ricompilazione>`

- Esempio:
  `make block`
- La compilazione sarà un po' più lenta, rispetto all'uso di `M=`
- Se, dopo aver utilizzato il make in questa forma, si modificano solo i sorgenti di qualche modulo dinamico, si può poi utilizzare di nuovo correttamente l'opzione `M=`

Analisi di `1-compilazione`

- Ora abbiamo tutte le conoscenze necessarie per capire il significato di tutti i passi che abbiamo eseguito in `1-compilazione`, nonché delle informazioni aggiuntive fornite nel file
- Studiamo quindi il documento assieme
  - A parte le informazioni pratiche di aiuto per problemi tecnici specifici
- Poi saremo pronti per creare il nostro primo modulo interno

Modulo interno

- Un modulo è interno se
  - i suoi sorgenti appartengono all'albero dei sorgenti del kernel
  - la sua compilazione è integrata nel sistema di kbuild
- Se un modulo interno è da (ri)compilare, e si esegue un `make modules`, o un semplice `make`, tale modulo viene (ri)compilato
  - Con il solo make, il kbuild controlla anche se il kernel è da ricompilare, e lo ricompila in caso affermativo
- Se un modulo interno è da (re)installare, e si avvia l'installazione dei moduli, allora tale modulo viene (re)installato
- Tutti i moduli presenti in un qualsiasi albero dei sorgenti ufficiale Linux sono moduli interni

Confronto moduli interni-esterni

- L’unica differenza tra un modulo interno ed uno esterno è che
  - Quello interno è integrato nell’albero dei sorgenti, secondo il senso spiegato nella slide precedente
  - Quello esterno è compilato a parte, nel modo che abbiamo già visto
- Ne segue quanto riportato nella prossima slide

Init ed exit

- Anche un modulo interno deve prevedere
  - Una funzione `init`
    - Eseguita al boot, se il modulo è *built-in*
    - Eseguita al caricamento del modulo, se il modulo è dinamico
  - Una funzione `exit`
    - Eseguita alla rimozione del modulo, e solo se il modulo è dinamico
- A meno di casi più complessi, che sono fuori dalla portata di questo corso base

Supporto kbuild modulo interno

- La compilazione del modulo è gestita dal Makefile presente nella subdirectory che contiene i sorgenti del modulo
- Il Makefile di livello superiore dovrà contenere una regola che farà eseguire il Makefile per il modulo, come visto nella sezione *Descending down in directories* in
  https://docs.kernel.org/kbuild/makefiles.html
  e con l'eccezione del top Makefile

Supporto kconfig modulo interno

- Una possibilità poco significativa è che la modalità di compilazione del modulo debba essere sempre la stessa, ossia sempre statico o sempre dinamico. A meno di tale possibilità, vale quanto segue:
  - Al modulo dovrà essere associata almeno una opzione di configurazione, definita in un file `Kconfig`
  - Tipicamente in un `Kconfig` presente nella stessa subdirectory in cui sono presenti i sorgenti del modulo

Esempio

- Analizziamo di nuovo il caso semplice già visto: `drivers/soc/microchip`
- Ricontrolliamo
  - Il contenuto del Makefile nella subdir
  - Il modo in cui l'invocazione ricorsiva del make è ottenuta, nel Makefile in `drivers/soc` e nel Makefile in `drivers`
  - Il Kconfig nella subdir
  - L'inclusione che avviene nei Kconfig in `drivers/soc` e in `drivers`
- Lasciamo stare il file .c, perché più complesso di quello che serve a noi

Domanda

- Alla luce di quanto detto finora, quali potrebbero essere i due modi di aggiungere un modulo interno, in termini di quali file e subdirectory aggiungere?

1. Creazione subdirectory dedicata

- Creare una subdirectory, contenente
  - File sorgenti
  - `Makefile` per il modulo
  - `Kconfig` per il modulo
- Agganciare il `Makefile` ed il `Kconfig` al `Makefile` e `Kconfig` di livello superiore
- Per ora assumiamo che tale subdirectory non sia creata come figlia diretta della directory radice
  - Gestiremo tale caso come esercizio

2. Integrazione in subdir esistente

- Scegliere una subdirectory in cui aggiungere i file sorgente del modulo
- Creare i file sorgente
- Estendere il `Makefile` nella subdirectory per compilare anche il modulo
- Estendere il `Kconfig` nella subdirectory per aggiungere anche le opzioni di configurazione del modulo

Domande propedeutiche

- Stiamo per creare il nostro primo modulo interno, con entrambe le precedenti soluzioni implementative. Inoltre compileremo il modulo sia built-in che dinamico
- Prima di iniziare l’esercizio, rispondiamo alle seguenti domande propedeutiche

Controllo init modulo

- Se il modulo interno
  - stampa semplicemente un messaggio alla sua inizializzazione, ed un messaggio alla sua rimozione
  - ed è compilato come un built-in

quale è l'unico messaggio che dobbiamo controllare, e dove lo dobbiamo cercare?

Risposta

- Controlliamo solo il messaggio di inizializzazione, perché la exit non sarà mai eseguita
- Lo cerchiamo nel kernel log

Ricompilazione per aggiunta modulo

- Dopo aver aggiunto un modulo dinamico al config del kernel, ossia dopo aver assegnato il valore `m` alla opzione che abilita la compilazione di tale modulo, quali operazioni bisogna effettuare per avere il nuovo modulo installato e caricabile correttamente?

Risposta

- Bisogna
  - Ricompilare e reinstallare sia il kernel che i moduli
    - Questo comporterà anche la compilazione del nuovo modulo
  - Riavviare
- Perché il core ed i moduli del kernel precedente
  - Potevano non includere strutture dati (o singoli campi extra), simboli o funzioni necessari per usare e fare funzionare il nuovo modulo
  - Potevano non combaciare col nuovo modulo in termini di configurazione di build

Esercizio: modulo con soluzione 1

- Aggiungere un modulo interno che stampa semplicemente un messaggio alla sua inizializzazione
- Aggiungere un modulo interno con la soluzione *1: subdirectory dedicata*
- Compilarlo come built-in
- Controllare che il modulo si inizializzi

- Per chi è in classe: se finite prima degli altri, alla slide 92 trovate una variante più difficile di questo esercizio

Soluzione

- Controllare il codice di tutti i file riportati nel file 2-moduli.txt, per la soluzione *1: subdirectory dedicata*
- Attivare dell'opzione tramite make menuconfig
- Per controllare l'inizializzazione

  1)Riavviare il nuovo kernel

  2)Cercare il messaggio di inizializzazione nel kernel log
- In caso di problemi, per controllare se il modulo è presente nel kernel, eseguite il target `modules_install` per fare aggiornare il file `modules-builtin`, e poi controllare che il vostro modulo sia presente nella lista

Nota su licenza modulo

- Se il modulo non fa nulla, nonostante sia presente in `modules_install`, il problema potrebbe essere che non avete aggiunto la dichiarazione della licenza
- Questo porta ad un fallimento silenzioso

Domanda

- Quale riga di codice fa sì che il modulo sia compilato come *built-in* o dinamico a seconda di quello che si sceglie nel .config?
- Come ottiene questo risultato?

Risposta 1/2

- La riga è `obj-$(CONFIG_HELLO_MODULE) += hello_module.o`
- Il valore della variabile `CONFIG_HELLO_MODULE` potrà essere `y`, `m` o vuoto. Nel Makefile quel valore viene sostituito testualmente all'espressione `$(CONFIG_HELLO_MODULE)`. Pertanto l'espressione composta `obj-$(CONFIG_HELLO_MODULE)` si trasforma in `obj-y`, `obj-m` oppure `obj-`

Risposta 2/2

- Quindi l'intero assegnamento fa sì che facendo sì che la parola `hello_module.o` venga concatenata alla variabile `obj-y`, `obj-m` oppure `obj-`
- Le variabili `obj-y` e `obj-m` contengono, rispettivamente, la lista dei file oggetto built-in e dei moduli dinamici. La lista obj- invece non è proprio considerata nel kbuild
- Quindi, concettualmente questa riga aggiunge il modulo hello_module alla lista dei moduli statici (`y`), oppure alla lista dei moduli dinamici (`m`) oppure ad una lista che non viene considerata affatto nel kbuild

Altre domande

- Nuove domande propedeutiche prima del prossimo esercizio

Ricompilazione per modifica config

- Se si commuta un modulo da built-in (`y`) a dinamico (`m`) nel config del kernel, quali operazioni bisogna effettuare per avere il nuovo modulo dinamico installato e caricabile correttamente?

Risposta

- Le stesse operazioni del caso di aggiunta del modulo
- Per gli stessi motivi visti per il caso di aggiunta modulo

Ricompilazione per rimozione

- Se si commuta un modulo da dinamico (`m`) a built-in (`y`), oppure se lo si elimina del tutto dalla configurazione del kernel, quali operazioni bisogna effettuare per eseguire un kernel in cui il modulo è built-in o assente?

Risposta

- Ovviamente le stesse operazioni dei casi precedenti, e per gli stessi motivi

Seconda parte esercizio

- Se necessario, estendere il codice del modulo affinché stampi un messaggio anche in uscita. Tale messaggio deve essere diverso da quello stampato in init
- Ricompilare il modulo come dinamico
- Installarlo
- Collaudarlo

Soluzione

- Il codice di tutti i file è sempre sempre nella soluzione 1 nel file *2-moduli.txt*
- Modificate l'opzione di configurazione del modulo da `y` a `m`, tramite, per esempio, `make menuconfig`
- Caricate il modulo con `modprobe`
  - E non con `insmod`, in accordo a quanto visto in https://medium.com/@adilrk/kernel-modules-86f07c93362a
- Rimuovete il modulo con `rmmod`
- Controllate che avvenga l'esecuzione di init ed exit: cercando i messaggi nel kernel log

Ricompilazione per modifica modulo

- Se si modifica (solo) il codice di un modulo interno, quali operazioni bisogna effettuare per arrivare ad avere la nuova versione del modulo reinstallata?

Risposta

- Basta solo
  - Ricompilare il modulo
  - Reinstallare i moduli

Esercizio, caso dinamico

- Modificate il messaggio stampato dall’init del modulo, e fate stampare il nuovo messaggio
  - Senza modificare il config
    - Il modulo rimane pertanto dinamico

Soluzione: caso dinamico

- Ricompilare i soli moduli, con
  `make M=<percorso-subdirectory>`
- Reinstallare i moduli
- Rimuovere il modulo se già caricato
- Ricaricare il modulo (con `modprobe`)
- Controllare che ci sia il nuovo messaggio nel kernel log
- **Non bisogna ricompilare, reinstallare e riavviare il kernel!**

Controllo versione modulo

- Come controllare se la versione correntemente installata di un modulo corrisponde all’ultima compilata?

Soluzione

- `ls -l \ /lib/modules/<versione_kernel_running>/kernel/<directory_del_modulo>/<nome_del_modulo>.ko`
- E confrontate la data di ultima modifica del file con la data di ultima modifica del file `.ko` del vostro modulo nel vostro albero di build

Esercizio, caso statico

- Modificate la configurazione del vostro modulo, da dinamico a statico
- Ricompilate il kernel (che scatenerà anche la ricompilazione del vostro modulo statico), reinstallate kernel e moduli, e riavvate
- Modificate di nuovo il messaggio stampato dall'init del modulo, e fate stampare il nuovo messaggio
- Che operazioni dovete svolgere in questo caso?

Soluzione: caso statico

- Dovete di nuovo ricompilare il kernel (scatenando anche la ricompilazione del vostro modulo statico), reinstallare il kernel e riavvare
- Molto più oneroso del caso dinamico

Esercizio: modulo con soluzione 2

- Implementate un nuovo modulo interno
  - Con lo stesso codice del modulo precedente, ma con messaggi diversi, per distinguerlo
  - Con la soluzione *2: integrazione in subdirectory esistente*
- Per semplicità, non scegliere una subdirectory in cui non sia già implementato qualche altro modulo
  - Per esempio scegliete drivers
  - E’ molto più semplice del caso di integrazione in una subdirectory che già implementa qualche modulo
- Configurate il modulo come preferite tra statico e dinamico, e collaudate il tutto

Soluzione

- Codice di tutti i file in 2-moduli, soluzione *2: integrazione in subdirectory esistente*
- I passaggi di compilazione, installazione e caricamento o ravvio sono gli stessi che abbiamo già visto per la soluzione 1

Varianti più difficili

- Le seguenti due varianti non sono richieste per l’esame
- Implementare il modulo con la soluzione *1: nuova subdirectory*, ma creando la subdirectory nella radice dell’albero dei sorgenti
- Implementare il modulo con la soluzione *2: integrazione in subdirectory esistente*, ma in una subdirectory in cui c’è già un modulo

Fine della presentazione per il modulo di kernel hacking dell’insegnamento di Sistemi Operativi

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli, e tra i moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Collegamento da modulo a core

- Il caricamento di un modulo dinamico prevede almeno due fasi principali
  - Semplice caricamento: l’immagine del modulo, che include la sezione testo (istruzioni) è caricata in memoria virtuale
  - Rilocazione: nella sezione testo (in memoria) ogni riferimento a simboli definiti nel core del kernel viene sostituito con l’effettivo indirizzo di tale simbolo nell’immagine del kernel in memoria

Collegamento da core a modulo

- Al contrario, non avviene alcuna rilocazione nel core del kernel, e in generale non avviene alcuna modifica all'immagine del kernel
- Il kernel invoca le funzionalità fornite dai moduli (sia statici che dinamici) solo tramite strutture di callback ossia di puntatori a funzione
- Le strutture sono definite staticamente all'interno di header file del kernel
- Quando un modulo viene inizializzato (all'avvio del kernel se modulo built-in, al caricamento se modulo dinamico), il codice di init del modulo definirà le istanze di callback appropriate per quel modulo e le registrerà con le funzionalità già previste staticamente dal codice del kernel

Collegamento da modulo a modulo

- I moduli possono interagire tra loro:
  - Tramite registrazione di callback, come si fa per permettere al kernel di interagire con un modulo
    - E tramite altri meccanismi indiretti che non menzioniamo
  - Tramite collegamento diretto dei simboli
    - Implementato mediante le macro `EXPORT_SYMBOL`
    - Si crea pertanto una dipendenza diretta se un modulo *A* utilizza un simbolo esportato da un modulo *B*
      - Sono proprio le dipendenze gestite automaticamente da `modprobe`
    - Con l'ausilio di funzionalità come `symbol_get` e `symbol_put` si possono gestire dipendenze opzionali

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Caricamento automatico moduli

- Supponiamo che
  - un certo modulo dinamico non sia caricato in un certo istante
  - in quell’istante in kernel debba eseguire una operazione per la quale è necessario tale modulo
- Se il kernel è in grado di capire che ha bisogno, appunto, di tale modulo, lo caricherà automaticamente

Esempio: cambio di I/O sched 1/4

- Controlliamo se gli I/O scheduler kyber e bfq sono configurati come built-in, controllando nel file /lib/modules/$(uname -r)/modules.builtin
- Se non lo sono, configuriamoli come built-in
  - Se è disponibile solo uno dei due nel menuconfig, configuriamo come modulo built-in l'unico dei due disponibile
- ricompiliamo, reinstalliamo e riavviamo
- Ricontrolliamo ed accertiamoci che siano entrambi built-in
- Ora, se controlliamo l'elenco degli I/O scheduler disponibili:
  cat /sys/block/<nome-vostro-device-storage>/queue/scheduler
  otteniamo (se ci sono sia kyber che bfq tra i built-in):
  [none] mq-deadline kyber bfq

Esempio: cambio di I/O sched 2/4

- Quanto segue può essere eseguito equivalentemente con kyber che con bfq
- Attiviamo bfq come I/O scheduler:
  echo bfq | sudo tee \
  /sys/block/<nome-vostro-device-storage>/queue/scheduler
- cat /sys/block/<nome-vostro-device-storage>/queue/scheduler
  none mq-deadline kyber [bfq]

Esempio: cambio di I/O sched 3/4

- Ora riconfiguriamo sia kyber che bfq come moduli dinamici, ricompiliamo, reinstalliamo e riavviamo
  - Come prima, se esiste solo uno dei due, lavoriamo solo su quello che esiste
- Controlliamo che né kyber né bfq compaiano più in modules.builtin
- Controlliamo con lsmod che non siano neanche tra i moduli caricati
- Adesso bfq e kyber non dovrebbero più apparire nella lista:
  cat /sys/block/<nome-vostro-device-storage>/queue/scheduler
  [none] mq-deadline

Esempio: cambio di I/O sched 4/4

- Ora, senza caricare il modulo di bfq, commutiamo su bfq
  echo bfq | sudo tee /sys/block/<nome-vostro-device-storage>/queue/scheduler
- cat /sys/block/<nome-vostro-device-storage>/queue/schedulernone mq-deadline [bfq]
- lsmod | egrep bfq
  bfq 106496 1
- Il modulo dinamico è stato caricato automaticamente!

Come è realizzato?

- Lo scopriremo da soli nel capitolo dedicato allo sviluppo del codice ...

Indice

- Meccanismo di kbuild
- Aggiunta di un primo modulo
  - Modulo esterno
  - Modulo interno
- Seconda parte, solo per il corso di kernel hacking della laurea magistrale
  - Collegamento tra core e moduli e tra moduli
  - Caricamento automatico di un modulo
  - Esercizio impegnativo

Esercizio impegnativo

- Il seguente esercizio è pensato per darvi l'occasione di espandere le vostre conoscenze
  - In particolare per coloro che stanno seguendo e finiscono man mano i vari esercizi di base molto prima dei propri colleghi
- Gli argomenti di tale esercizio sono aggiuntivi rispetto al corso
  - Non sono richiesti in alcun modo all'esame
- Per chi sta seguendo: se tale esercizio diventa una distrazione dal seguire gli argomento di base, ne otterrete un danno formativo anziché un beneficio

Servizi periodici

- Ci sono una serie di servizi che il kernel deve eseguire periodicamente
  - Controllare, per ciascun task in esecuzione, se è finita la fetta di tempo in cui tale task doveva restare in esecuzione
  - Controllare se è arrivato il momento di eseguire delle chiamate di funzione che erano state accodate per essere eseguite in un dato istante nel futuro
  - Aggiornare il cosiddetto tempo di sistema (vedi prossima slide)
- A questo scopo, il kernel programma un timer hardware per fargli generare una richiesta di interruzione periodica
  - Questa richiesta periodica viene chiamata *tick*
  - La routine di servizio di tale interrupt esegue i servizi elencati sopra

jiffies e frequenza del *tick*

- Ad ogni *tick* viene incrementata di una unità una variabile globale chiamata *jiffies*
  - I *jiffies* misurano quindi il tempo di sistema
- La frequenza con di arrivo del *tick* è memorizzata in una costante globale, chiamata HZ
- Ad ogni *tick*, è necessario un tempo non infinitesimo per eseguire i servizi periodici
  - Pertanto il *tick* non potrà essere troppo frequente, altrimenti un core della CPU passerebbe troppo tempo, se non tutto il tempo, ad eseguire i servizi periodici
- I *jiffies* quindi non hanno una granularità finissima
- Quasi sempre nella CPU sono disponibili anche altri timer, ad alta risoluzione
  - Si utilizzano quelli quando si vuole avere molta precisione

Modifica del valore di HZ

- Il valore della costante HZ è modificabile attraverso il config
- Ma non direttamente
  - si può solo selezionare un valore all’interno di un insieme di valori prestabilito

Esercizio

- Fare stampare il valore di HZ dal modulo quando viene inizializzato
- Provare tre esecuzioni del kernel
  - Con HZ originale
  - Con HZ diminuito
  - Con HZ aumentato
- Per ciascuna di tali esecuzioni, oltre a verificare il valore di HZ stampato dal modulo, valutare l’overhead del kernel a sistema idle, e latenza del kernel
  - Confrontare i valori ottenuti e valutare cosa si guadagna e cosa si perde variando il valore di HZ

Semplice misura overhead totale

- Con, ad esempio, il comando top, controllare l’utilizzazione della CPU al variare di HZ
  - In particolare, controllare le percentuali di tempo che la CPU passa nei vari stati (usr, sys, idle, wait, …)

Strumenti di misura della latenza

- Misura risposta ad interrupt: cyclictest
  https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/start
  https://man.archlinux.org/man/cyclictest.8.en
- Misura tempi di bloccaggio dei task
  - Panoramica
    https://www.brendangregg.com/blog/2016-02-01/linux-wakeup-offwake-profiling.html
  - Pagina di manuale:
    https://manpages.debian.org/testing/bpfcc-tools/offwaketime-bpfcc.8.en.html
  - Istruzioni dettagliate per l'installazione:
    - Nel documento *offwaketime*, all'interno del materiale didattico